

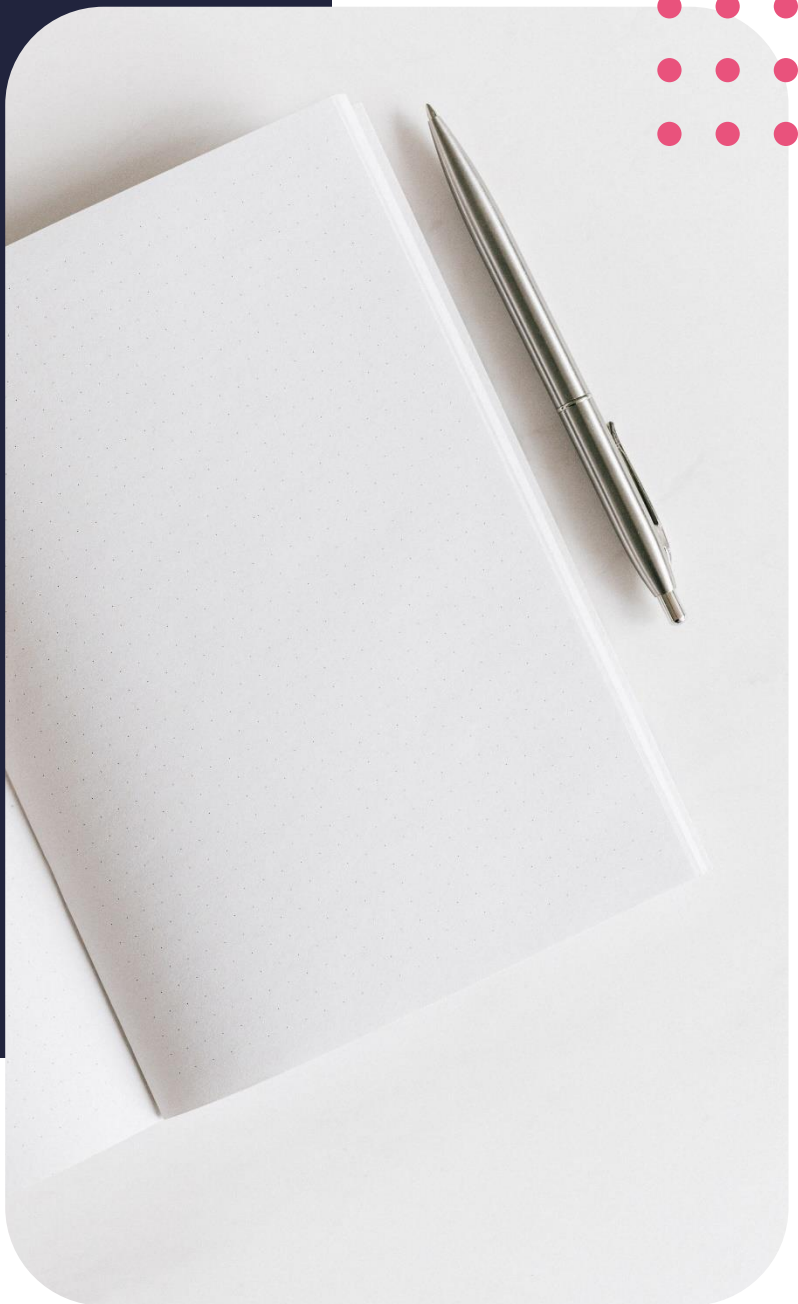
Diploma in Data Analytics

Dates and times continued



Contents

3	Introduction
3	Lesson outcomes
3	Dates and times continued
4	Dates and times essentials
6	Intro to time series analysis
8	References



Lesson outcomes

By the end of this lesson, you should be able to:

- Confidently deal with data containing inconsistent date and time formats
- Understand basic concepts surrounding time series analysis

Introduction

To extract useful information from a dataset, the data needs to be in uniform formats. In order to extract information from data, we need to transform the data into a format that is usable. Whatever we call it- data transformation, data munging, or data wrangling- this step is unavoidable and a core skill to the data analysts' toolkit. The real world will hardly ever present an unblemished data set, it is therefore up to us to transform this data into a format where we can use it to extract the insights, we need from it.

In this lesson, we will continue to broaden our understanding of dealing with dates and times in R. Many datasets contain dates and times and we need to make the step of data handling dates and times as simple and effective in our data analytics arsenal as possible. Therefore, we will continue building on dates and times data wrangling throughout a large part of this lesson. We will end the lesson by introducing time series analysis concepts.

Dates and times continued

Dates and times recap

In the previous lesson, we were introduced to working with dates in R through some basic commands that we can use to extract information about today's date, time and time zone as well as some ways of transforming date formats using the lubridate package in R.

- We learnt that in order to get the current date and time particulars, you can use `sys.timezone` from R to extract information about where your current timezone is located.
- The function `sys.date` returns the current date.
- The function `sys.time` returns the current date and time for your timezone.
- Alternatively, using the lubridate package, we use the command `now` to offer us the same particulars as the function `sys.time`.
- The `as.date()` function can convert a string that is in date format to a date object.
- If we are using the lubridate package, we can just use the command `ymd` or `mdy` depending on the format of the date to convert the string to a date object.

Did you know: R date of origin

In the same way that excel has an origin date, 1 January 1900, that it starts counting from, so does R. R's date that it starts counting at, is 1 January 1970.

Date objects are stored in R as integer values, allowing for dates to be compared and manipulated as you would a numeric vector. When we refer to dates, earlier dates are “less than” later dates. Dates are represented as the number of days since 1970-01-01, with negative values for earlier dates.

Types of dates and times data in R

R contains three types of date/time data that it uses to declare an instant in time:

- Date
 - Tibbles print this as <date>.
- Time
 - Tibbles print this as <time>.
- Date-time
 - Tibbles print this as <dtm>. R also refers to this format as POSIXct.

Lubridate

Lubridate forms a part of the tidyverse library. This means that lubridate integrates well with other packages that forms a part of the tidyverse library. Lubridate simplifies the process of working with dates and times in R. Regardless of if the dates are stored in date objects, time series objects or POSIXct objects, the lubridate package has one function that will treat all of these different storage formats uniformly.

Parsing dates and times

As we have already mentioned, getting dates in the correct format can be challenging.

- We have learnt that the functions ymd, mdy and dmy can parse the date and are able to handle an extensive array of formats and separators, that will ease the parsing process considerably.
- Lubridate is also able to handle information containing dates and times, by simply adding hms() or ms() to the aforementioned statements.
 - For example: ymd_hms(), ymd_ms()

Duration between dates and times

Lubridate is also able to get the duration of time between two instances

Functions for working with durations include:

- is.duration()
- as.duration()
- duration()
- dseconds()
- dminutes()
- dhours()
- ddays()
- dweeks()
- dyears()

If, for instance, we want to know how much time occurred between the arrival and departure of a patient, we would be able to achieve that with the use of the duration command.

Setting and extracting information

We are also able to set and extract information from dates and time with lubridate using the functions second, minute, hour and so forth. We can use these functional to set the specified information. This will alter the date and time.

Dates and times distributed across multiple columns

Sometimes date information is not represented by a single string and spread across multiple columns.

We can use the make_date function for dates or the make_datetime function for date-times to create a singular product from information spread across multiple columns.

Rounding dates and times

- The round_date function will round the date-time object to the nearest value specified.
- If the date-time is exactly halfway between two consecutive units, the convention is to round up.
- Function ceiling_date will round the date-time object up to the nearest value of the specified time-date value.
- Function floor_date will round the date-time object down to the nearest value of the specified time-date value.

Intervals

Another useful functionality of the package lubridate, is the ability to indicate the time between two instances occurring with the help of interval functions. Intervals provide a way to change convert between periods and durations whilst maintaining all information about a period.

Functions for operating with intervals include:

- is.interval()
- as.interval()
- interval()
- int_shift()
- int_flip()
- int_aligns()
- int_overlaps()
- %within%

Setting components

Furthermore, we are able to use each assessor function, that includes functions like year, month, mday, hour and so forth, to set the elements of a date-time.

Alternatively, if we don't want to change the existing date-time, we are able to create a new date-time with the function update().

Intro to time series analysis

A variable that is measured over time in sequential order is called a time series. It may consist of week production inputs, monthly sales, annual rainfall or any other variable for which the value is observed at regular intervals.

Through the analysis of data that span over a period of time, we are able to spot trends and patterns and in these will aid us to recognize variations from the past in order to more accurately predict future instances.

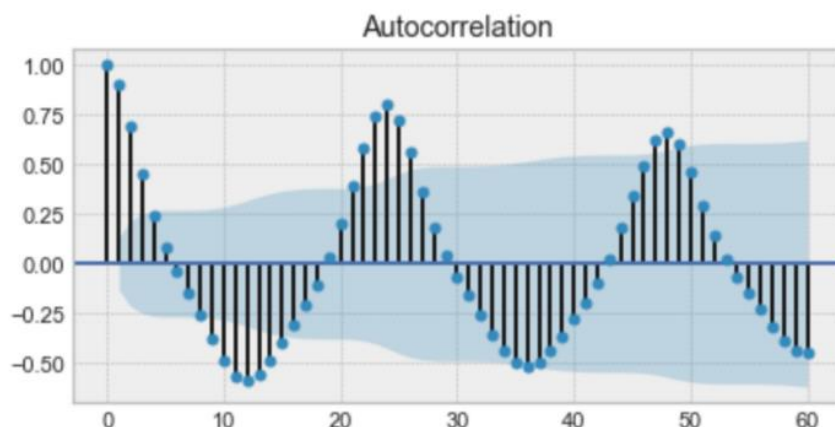
Why time series

Time series can help to understand trends of past business behaviour better that will in turn help make better predictions for the future business needs. We understand what the business need for electricity might be during certain seasons in the year and plan better for future seasons electricity consumption.

Other questions that arise when dealing with time series trends, are if the trend is stationary, if there's a seasonal component involved with the trend or if the target variable is autocorrelated?

Autocorrelation

Autocorrelation is also known as the degree of similarity between observations as a function of the time lag amongst them.



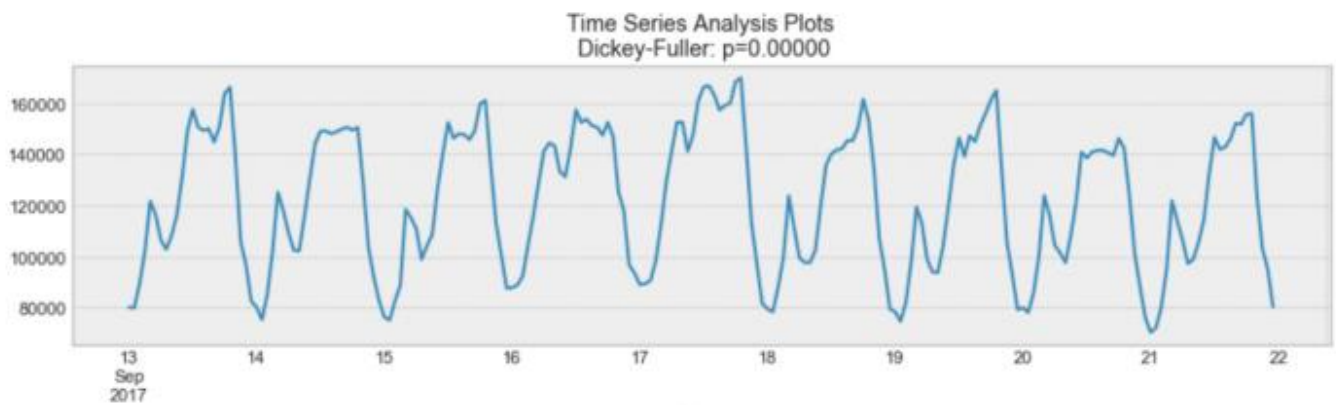
If we look at the autocorrelation plot, it shows us that the first and 24th value is highly correlated. So is the 12th and the 36th observations. This shows us that every 24th observation will have a degree of similarity. The sinusoidal wavelength of the plot is also an indication of seasonality in the data.

Seasonality

Seasonality refers to the existence of variations that take place at specific methodical intervals less than a year apart, for instance weekly, monthly, or quarterly.

If we refer back to the electricity example again, we expect the electricity consumption to be higher during the day than it is during the night, so this would be a case of daily seasonality.

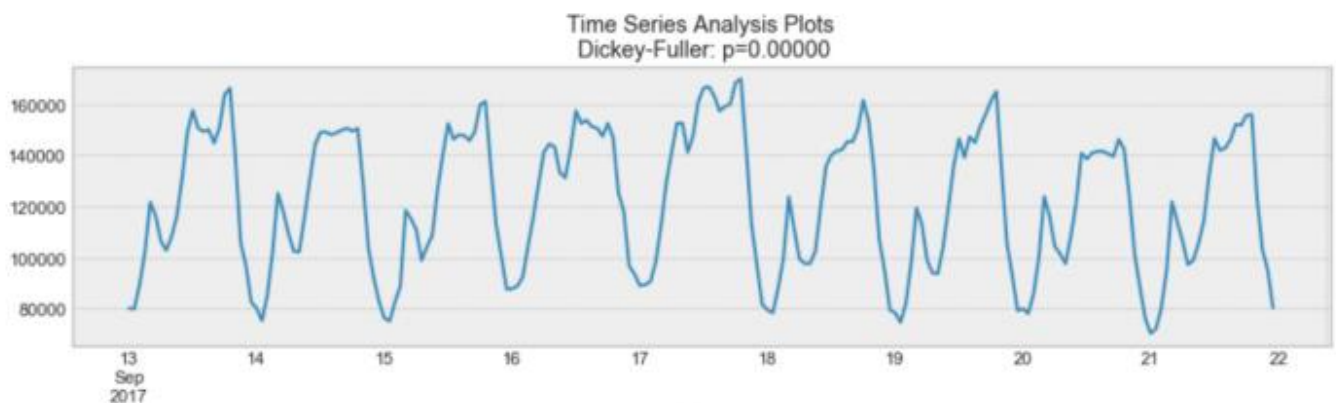
We also expect electricity consumption to be higher in winter season than in summer season.



The sinusoidal shape of the autocorrelation plot, like we mentioned, is an indication to us of seasonality in the data.

Stationary

Time series is assumed to be stationary if the mean, variance and covariance does not change over time.



This plot is an image of a stationary process. The mean and variance stays the same over time. The ideal is to have a stationary time series for modelling purposes, but of course, like in the case of stock prices, this is not always possible. We can apply transformations to the time series if it is not stationary, but that we will discuss in the next lesson.

Regression techniques

Let us look at the many ways there are to model time series in order to make predictions about the data.

Regression techniques that can be used for time series data include:

- Moving averages
- Seasonal analysis
- Exponential smoothing
- ARIMA models

In the next lesson we will delve a bit deeper into each of these models and understand when to use each respective model.

Additional resources

- For more reading and information regarding the functionality of the lubridate package, you can visit the R for Data Science book by Hadley and Golemund created or this R-project link: <https://cran.r-project.org/web/packages/lubridate/lubridate.pdf>
- Link for data set mentioned during demo: <https://www.kaggle.com/saurograndi/airplane-crashes-since-1908>

References

- Banghart, M., 2019, *Data Wrangling Essentials*, Supporting Statistical Analysis for Research,
- Beck, C., 2012, Handling date-times in R, Department of Biostatistics Vanderbilt University School of Medicine, <http://biostat.mc.vanderbilt.edu/wiki/pub/Main/ColeBeck/datestimes.pdf>
- Garrett Golemund, Hadley Wickham (2011). Dates and Times Made Easy with lubridate. Journal of Statistical Software, 40(3), 1-25. <http://www.jstatsoft.org/v40/i03/>.
- Gorman, B., 2018, *Dates and times in R without losing your sanity*, GormAnalysis, <https://www.gormanalysis.com/blog/dates-and-times-in-r-without-losing-your-sanity/>
- Golemund, G., 2012, *Do more with dates and times in R with lubridate 1.1.0*, R-statistics, <https://www.r-statistics.com/2012/03/do-more-with-dates-and-times-in-r-with-lubridate-1-1-0/>
- Golemund, G., 2020, *ceiling_date function*, R Documentation, https://www.rdocumentation.org/packages/lubridate/versions/0.2.3/topics/ceiling_date
- Golemund, G., Wickham, H. & Spitnu, V., 2011, *Dates and times made easy with lubridate*, tidyverse, <https://lubridate.tidyverse.org/reference/lubridate-package.html>
- Golemund, G., Wickham, H., 2017, *R for Data Science*, O'Reilly, <https://r4ds.had.co.nz/dates-and-times.html>
- <https://www.ssc.wisc.edu/sscc/pubs/DWE/book/5-5-date-and-time-variables.html>
- Kabacoff, R.I., 2017, *Date Values*, Quick-R by DataCamp, <https://www.statmeth.ods.net/input/dates>
- Peixeiro, M., 2019, *The Complete Guide to Time Series Analysis and Forecasting*, towards data science, <https://towardsdatascience.com/the-complete-guide-to-time-series-analysis-and-forecasting-70d476bfe775>
- UCLA: Statistical Consulting Group, 2020, How does R handle date values?, UCLA Institute for digital research & education, <https://stats.idre.ucla.edu/r/faq/how-does-r-handle-date-values/#:~:text=When%20R%20looks%20at%20dates,origin%20is%20January%201%2C%201970.>